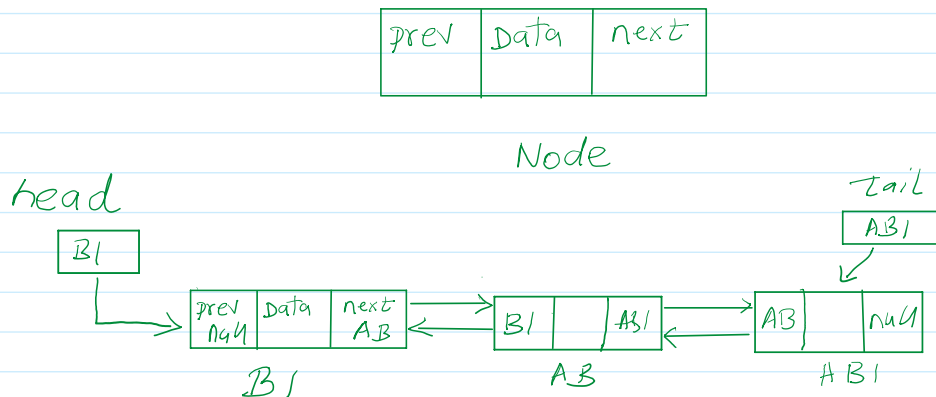


Doubly Linked List

05 September 2026 18:43

A doubly linked list is a linear data structure in which each node contains a references to both the previous and the next node. It supports traversal in both forward and backward direction.



In doubly linked list, the first node is called the head and the previous pointer is null. The last node is called the tail and the next pointer is NULL. Each intermediate node is connected to both its previous and next nodes.

For example, a music playlist application can use a doubly linked list to move to the previous or next song without restarting the playlist.

Characteristics of a doubly linked list

A doubly linked list has the following characteristics:

- Each node contains one data fields and 2 pointers or reference variables.
- It supports traversal in both forward and backward direction.
- The next pointer or the next reference of the last node is null.
- It allows efficient insertion and deletion when the node location is known.

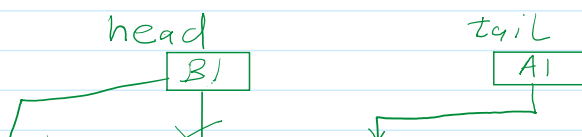
How it works?

Head and Tail: The head links to the first data of the list and the tail links to the last data. These links enhances operations such as adding or removing elements at the head or at the end of the list.

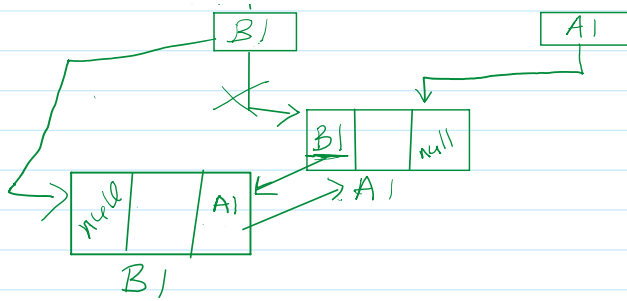
Traversal: It can start from the head and follows the next pointer to move forward or start from the tail and follows the Prev pointer to move backward.

Insertion and deletion: Adding or deleting notes can be done efficiently at any point in the list.

Insertion at head



```
newNode.setNext(head);  
head = newNode;  
public void setNext(DoubleNode node){  
    this.next = node;  
}
```



```

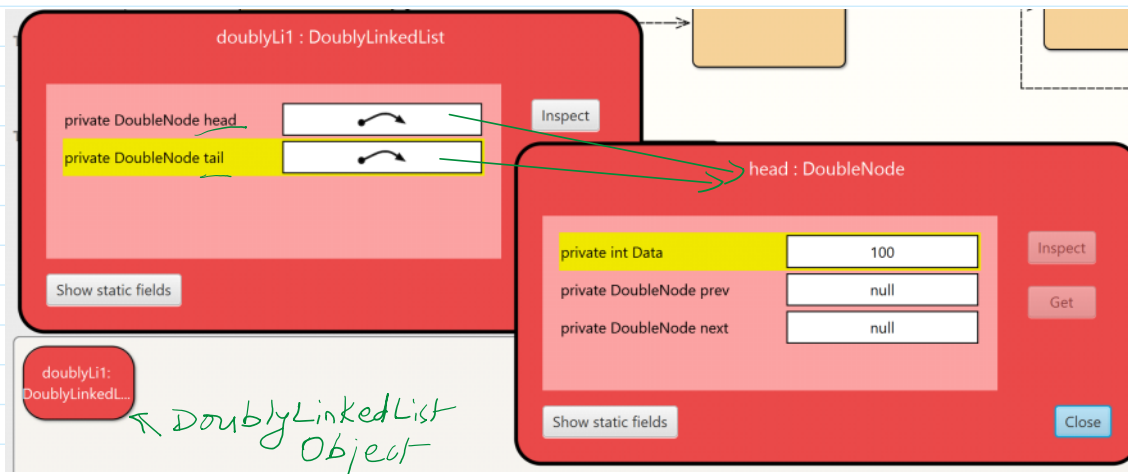
    this.next = node;
}

```

```

public void insertNodeAtHead(int data){
    DoubleNode newNode = new DoubleNode(data);
    if(head == null){
        head = tail = newNode;
        return;
    }
    head.setPrev(newNode);
    newNode.setNext(head);
    head = newNode;
}

```



```

public void traverseFromTail(){
    if(tail == null){
        System.out.print("\nList is empty ");
        return;
    }
    System.out.print("\nList traversal from tail: \n");
    for(DoubleNode t = tail; t != null; t = t.getPrev()){
        System.out.print(t);
    }
    System.out.print("\n");
}

```

```

package LinkedList;
/**

```

```

* Write a description of class DoublyLinkedList here.
*
* @author (your name)
* @version (a version number or a date)
*/
public class DoublyLinkedList
{
    private DoubleNode head, tail;

    public DoublyLinkedList(){
        System.out.print("\nMemory is allocated for Doubly Linked
list.");
        head = tail = null;
    }

    public void insertNodeAtHead(int data){
        DoubleNode newNode = new DoubleNode(data);
        if(head == null){
            head = tail = newNode;
            return;
        }
        head.setPrev(newNode);
        newNode.setNext(head);
        head = newNode;
    }

    public void insertNodeAtTail(int data){
        DoubleNode t = new DoubleNode(data);
        if(head==null){
            head = t;
            tail = t;
        }
        else{
            tail.setNext(t);
            t.setPrev(tail);
            tail = t;
        }
    }

    public void insertNodeAfterANode(int node, int data){
        if(head == null){
            System.out.print("List is Empty ");
        }
        else{
            for(DoubleNode t = head; t!=null; t=t.getNext()){
                if(t.getData() == node){
                    DoubleNode n = new
DoubleNode(data);
                    if(t.getNext() !=null){
                        n.setNext(t.getNext());
                        t.getNext().setPrev(n);
                    }
                    else{
                        tail = n;
                    }
                }
            }
        }
    }
}

```

```

        }
        t.setNext(n);
        n.setPrev(t);
        return;
    }
}
System.out.print("\nNode Not Present in the List ");
}
}

public void insertNodeBeforeANode(int node, int data){
    if(head == null){
        System.out.print("List is Empty ");
    }
    else{
        for(DoubleNode t = head; t!=null; t=t.getNext()){
            if(t.getData() == node){
                DoubleNode n = new
DoubleNode(data);
                if(t.getPrevious() !=null){
                    n.setPrev(t.getPrevious());
                    t.getPrevious().setNext(n);
                }
                else{
                    head = n;
                }
                t.setPrev(n);
                n.setNext(t);
                return;
            }
        }
        System.out.print("\nNode Not Present in the List ");
    }
}

public void deleteFromHead(){
    if(head == null){
        System.out.print("\nList is Empty ");
    }
    else{
        System.out.print("\nNode Deleted Successfully : " +
head.getData());
        head = head.getNext();
        head.getPrevious().setNext(null);
        head.setPrev(null);
    }
}

public void deleteAGivenNode(int node){
    if(head == null){
        System.out.print("\nList is Empty ");
    }
    else{
        for(DoubleNode t = head; t!=null; t=t.getNext()){

```

```

        System.out.println("List is Empty ");
    }
    else{
        for(DoubleNode t = head;t!=null;t= t.getNext()){
            if(t.getData() == node){
                → if(t.getPrevious() == null){
                    deleteFromHead();
                }
                → else if(t.getNext() == null){
                    deleteFromTail();
                }
                else{
                    → System.out.print("\nNode Deleted Successfully : " + t.getData());

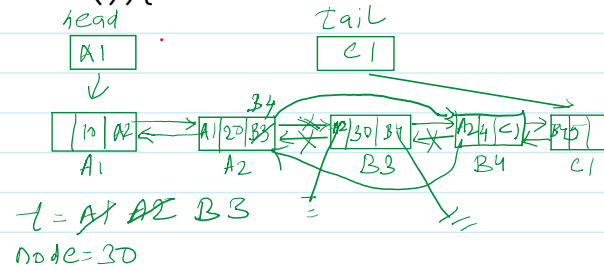
                    t.getPrevious().setNext(t.getNext());
                    t.getNext().setPrev(t.getPrevious());
                    t.setPrev(null);

                    t.setNext(null);
                }
            }
            return;
        }
        System.out.print("\nNode Not Present ");
    }
}

public void deleteFromTail(){
    if(tail == null){
        System.out.print("\nList is Empty ");
    }
    else{
        System.out.print("\nNode Deleted Successfully : " + tail.getData());
        tail = tail.getPrevious();
        tail.getNext().setPrev(null);
        tail.setNext(null);
    }
}

public void traverseFromHead(){
    if(head == null){
        System.out.print("\nList is empty!");
        return;
    }
    System.out.print("\nList traversal from start: \n");
    for(DoubleNode t = head;t != null; t = t.getNext()){
        System.out.print(t);
    }
    System.out.print("\n");
}
//backward traversing
public void traverseFromTail(){
    //write your code here

```



```

        if(tail == null){
            System.out.print("\nList is empty ");
            return;
        }
        System.out.print("\nList traversal from tail: \n");
        for(DoubleNode t = tail; t != null; t = t.getPrevious()){
            System.out.print(t);
        }
        System.out.print("\n");
    }
}

import java.util.*;

/**
 * Write a description of class Main here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Main
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        int data, node;
        DoublyLinkedList obj = new DoublyLinkedList();
        for(;;){
            System.out.print("\n===== " +
                "\nChoose A Following Option : " +
                "\n1. Insert Node At Head " +
                "\n2. Insert Node At End " +
                "\n3. Insert Node After A Node " +
                "\n4. Insert Node Before A Node " +
                "\n5. Delete Node At Head " +
                "\n6. Delete Node At End " +
                "\n7. Delete A Given Node " +
                "\n8. Traverse From Head " +
                "\n9. Traverse From Tail " +
                "\n10. Exit " +
                "\n===== \n\n");
            int ch = sc.nextInt();
            switch(ch){
                case 1:
                    System.out.print("\nEnter Data to be Inserted At
Head: ");
                    data = sc.nextInt();
                    obj.insertNodeAtHead(data);
                    break;
                case 2:
                    System.out.print("\nEnter Data to be Inserted At
End: ");
                    data = sc.nextInt();
                    obj.insertNodeAtTail(data);
                    break;
            }
        }
    }
}

```

```

        case 3:
            System.out.print("\nEnter Data to be Inserted : ");
            data = sc.nextInt();
            System.out.print("\nEnter the Data After which it
will be Inserted : ");
            node = sc.nextInt();
            obj.insertNodeAfterANode(node, data);
            break;
        case 4:
            System.out.print("\nEnter Data to be Inserted : ");
            data = sc.nextInt();
            System.out.print("\nEnter the Data Before which it
will be Inserted : ");
            node = sc.nextInt();
            obj.insertNodeBeforeANode(node, data);
            break;
        case 5:
            obj.deleteFromHead();
            break;
        case 6:
            obj.deleteFromTail();
            break;
        case 7:
            System.out.print("\nEnter Data to be Deleted : ");
            data = sc.nextInt();
            obj.deleteAGivenNode(data);
            break;
        case 8:
            obj.traverseFromHead();
            break;
        case 9:
            obj.traverseFromTail();
            break;
        case 10:
            System.exit(0);
    }
}
}
}

```

```

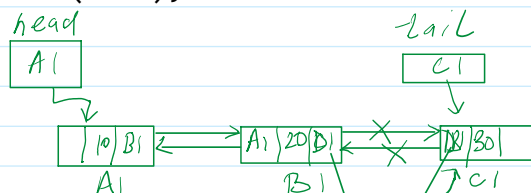
for(DoubleNode t = head; t!=null; t=t.getNext()){
    if(t.getData() == node){
        if(t.getNext() == null){ //case of last node
            insertNodeAtTail(data);
            return;
        }

```

```

→ DoubleNode n = new DoubleNode(data);
n.setPrev(t);
n.setNext(t.getNext());
n.getNext().setPrev(n);
t.setNext(n);
return;

```

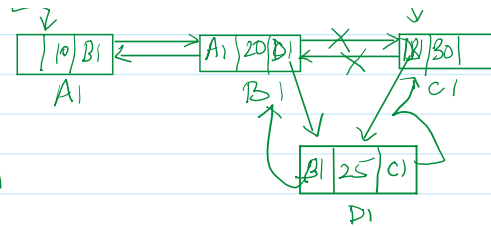


```

    ...getNext(), return t;
    t.setNext(n);
    return;
}
}

```

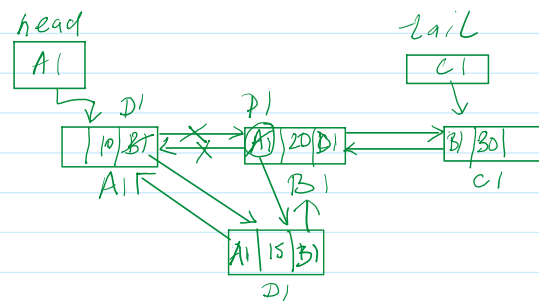
$t = B1$
 $n = D1$



```

for(DoubleNode t = head; t!=null; t=t.getNext()){
    if(t.getData() == node){
        if(t.getPrevious() ==null){
            insertNodeAtHead(data);
            return;
        }
        DoubleNode n = new DoubleNode(data);
        n.setNext(t);
        n.setPrev(t.getPrev());
        t.getPrev().setNext(n);
        t.setPrev(n);
        return;
    }
}
System.out.print("\nNode Not Present in the List ");
}

```



$t = B1$
 $n = D1$